ensil-ensci ÉCOLE D'INGÉNIEURS DE LIMOGES

TP de Traitement d'Image – MIX5

Ouiddad Labbani-I./Anis Debbah

1 Introduction

L'objectif de ce TP est de vous faire réaliser un ensemble de traitements numériques pour tester l'efficacité de certains algorithmes de traitement d'image et de les adapter en fonction de l'application et du résultat attendu.

Nous utiliserons le langage **Python** ainsi que les librairies standards de l'industrie : **OpenCV**, **NumPy** et **Matplotlib**.

Une première partie de ce TP sera consacrée à la prise en main de ces bibliothèques à travers quelques exemples. Votre travail commencera juste après et il vous sera demandé de produire un ensemble de traitements pour rectifier/extraire des informations d'images (celles fournies dans le dossier).

Le **compte-rendu** est à rendre dans un délai d'une semaine à compter de la fin des séances. Il doit inclure vos résultats, mais également une **explication du raisonnement utilisé** pour chaque chaine de traitements et dans chaque cas. Une version numérique (.zip), contenant le compte-rendu (au format .pdf) et les scripts Python (.py) ou le Notebook Jupyter (.ipynb), doit être envoyée à :

`anis.debbah@unilim.fr`

Objet du mail :

`CR_TP_ImageProcessing_votreNom`

Contenu du mail :

`TP_ImageProcessing_votreNom.zip`

2 Mise en place de l'environnement

Pour ce TP, nous vous recommandons fortement d'utiliser l'IDE **Spyder**. Son interface intègre un éditeur de code, une console, et surtout un **explorateur de variables**.

3 Mise en place de l'environnement (Windows)

Afin d'éviter les conflits de versions fréquents sur Windows, nous allons suivre une procédure d'installation stricte.

Étape 1 : Installation de Python (Microsoft Store)

Cette méthode est recommandée car elle configure automatiquement les variables systèmes.

1. Ouvrez le **Microsoft Store** (Menu Démarrer > Store).
2. Cherchez **"Python 3.13"** (ou la version la plus récente disponible).
3. Cliquez sur **"Obtenir"** ou **"Installer"**.

Étape 2 : Installation de Spyder et des librairies

Une fois Python installé, ouvrez votre invite de commande (*cmd* ou *PowerShell*) et exécutez la commande suivante pour tout installer :

```
python -m pip install --upgrade spyder numpy matplotlib opencv-python
```

Attendez que l'installation soit complète.

Étape 3 : Lancement et Stabilisation (Important)

Lancez Spyder.

Correction préventive : Si vous rencontrez des erreurs au lancement (type *ImportError : numpy.core.multiarray* ou *matplotlib*), exécutez impérativement la commande suivante **directement dans la console IPython de Spyder** (en bas à droite) :

```
!pip install --force-reinstall numpy==1.24.3 matplotlib==3.7.1
```

Une fois cette commande terminée, **redémarrez le noyau** de Spyder.
Une fois l'installation terminée, lancez Spyder.

4 Découverte des outils (OpenCV & NumPy)

Pour commencer, il faut importer les librairies. Voici **quelques exemples** à tester (dans un script ou un Notebook) :
Pré-requis :

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

### (Note aux étudiants. Attention, OpenCV lit les images en BGR, Matplotlib
affiche en RGB.)
```

Ex.1 : **Lecture et Affichage**

```
flowers = cv2.imread('flowers8.png')
# Conversion BGR -> RGB pour l'affichage correct
flowers = cv2.cvtColor(flowers, cv2.COLOR_BGR2RGB)
print(flowers.shape, flowers.dtype)
```

```
grey = cv2.cvtColor(flowers, cv2.COLOR_RGB2GRAY)
plt.imshow(grey, cmap='gray')
plt.show()

# Accès pixel (Indexation à 0 en Python !)
pix = flowers[275, 317, :]
print(pix)
```

Ex.2 : **Histogrammes et Normalisation**

```
im = cv2.imread('image_de_votre_choix.png')
im = cv2.cvtColor(im, cv2.COLOR_BGR2RGB)

# Normalisation en flottant (0.0 - 1.0)
im_double = im.astype(float) / 255.0

# Histogramme (aplatir l'image avec ravel)
hist_vals, bins = np.histogram(im_double.ravel(), bins=256, range=(0,1))

plt.figure(figsize=(6,8))

# L'image
plt.subplot(2,1,1)
plt.imshow(im_s)
plt.title("Image contrastée")
plt.axis("off")

# L'histogramme
plt.subplot(2,1,2)
plt.plot(hist_vals)
plt.title("Histogramme")
plt.show()

# Etirement de contraste (Percentile)
p2, p98 = np.percentile(im_double, (2, 98))
im_s = np.clip((im_double - p2)/(p98 - p2), 0, 1)
plt.imshow(im_s)
```

Ex.3 : **Espaces couleurs (Green Screen)**
En Python, on calcule manuellement les coordonnées chromatiques ($r = R/\Sigma, g = G/\Sigma$).

```
subject = cv2.imread('greenscreen.jpg').astype(float)
# Calcul de l'intensité (Somme R+G+B)
S = np.sum(subject, axis=2)
S[S==0] = 1.0 # Eviter division par zero

# Coordonnée g (Attention: BGR -> G est l'indice 1)
g = subject[:,:,1] / S
```

```
mask = g < 0.45
plt.imshow(mask, cmap='gray')

# Application du masque (Multiplication matricielle)
mask_3d = np.stack([mask]*3, axis=2) # On étend le masque en 3D
res = subject * mask_3d
plt.imshow(res.astype(np.uint8))
```

Ex.4 : **Convolution**

```
# Noyau moyenneur
K = np.ones((21,21), np.float32)/(21**2)
lena = cv2.imread('lena.png')

# Convolution 2D
out = cv2.filter2D(src=lena, ddepth=-1, kernel=K)
plt.imshow(cv2.cvtColor(out, cv2.COLOR_BGR2RGB))

# Noyau Gaussien
sigma = 5
k_size = int(2*np.ceil(3*sigma)+1)
gauss_1d = cv2.getGaussianKernel(k_size, sigma)
K_gauss = gauss_1d @ gauss_1d.T # Produit matriciel pour noyau 2D
out_g = cv2.filter2D(lena, -1, K_gauss)
```

5 Travail à réaliser

5.1 Exercice préliminaire

Commençons par considérer le cas simple de l'image `'shapes.png'`.

1. En analysant l'histogramme de l'image ou en convertissant l'image, identifiez les caractéristiques colorimétriques des différents éléments.
2. Proposez alors un traitement permettant de sélectionner uniquement les objets bleus sous la forme d'un masque binaire.
3. Adaptez votre approche pour ne conserver que le cercle (vous pouvez utiliser `cv2.connectedComponentsWithStats` ou `cv2.findContours`).

5.2 Exercice 2

On considère maintenant les images `'cas_1.png'`, `'cas_2.png'`, et `'cas_3.png'`. Chacune devrait vous poser une difficulté spécifique (bruit, contraste, etc.). A vous de trouver la parade pour chaque situation en utilisant les filtres adaptés (Gaussien, Médian, Égalisation d'histogramme, etc.) !

5.3 Exercice 3

On considère cette fois-ci l'image `'yellowtargets.png'`.

1. Proposez une technique pour binariser cette image et extraire uniquement les "motifs jaunes" (Indice : L'espace HSV peut être utile via `cv2.cvtColor` avec `cv2.COLOR_BGR2HSV`).
2. Proposez, si besoin, des traitements adaptés pour supprimer le bruit de fond et combler les trous de l'objet (Opérations morphologiques : érosion, dilatation).
3. Proposez une méthode pour extraire les contours de ces patterns.